NAME:Yugandhar

REGNO:192472353

COURSE CODE/COURSE:CSA0606/DAA

ASSESSMENT RECORD
Trade-off Analysis of Recursive and Iterative Merge Sort

1. State the Assessment Question
Evaluate the trade-offs between recursive and iterative implementations of Merge Sort. Implement both approaches, record their execution time and memory usage, analyze stack usage and function-call overhead, interpret the experimental results, and justify which approach is more efficient for practical applications.

Problem Statement
Merge Sort is a divide-and-conquer sorting algorithm with O(n log n) time complexity. It can be implemented recursively, where the array is repeatedly divided through function calls, or iteratively, where the same merge operations are controlled using loops. The purpose of this assessment is to compare both implementations using the same input data and determine the practical trade-offs in execution time, memory usage, stack usage, and function-call overhead.

Input Requirements
- An array of integers to be sorted.
- The same array sizes and values should be used for both implementations.
- Recommended test sizes: 1,000; 5,000; 10,000; 50,000; and 100,000 elements.
- Randomly generated values can be used to avoid input-order bias.

Expected Output
- The array must be sorted in ascending order by both implementations.
- Execution time for recursive and iterative Merge Sort should be recorded.
- Memory usage should be recorded or estimated for both implementations.
- Stack usage and function-call overhead should be discussed.
- A comparison should identify the more practical implementation.

Constraints

- Both methods must implement Merge Sort rather than using a built-in sorting function.
- Both methods must sort identical copies of each test input.
- The comparison should be performed under the same hardware/software conditions.
- Timing should be measured over sufficiently large inputs because very small inputs may give unstable measurements.
- Theoretical complexity should be considered along with measured experimental results.

Assessment Criteria

Criterion	Expected Evidence
Correctness	Both methods produce the same correctly sorted output.
Implementation	Recursive and iterative Merge Sort are implemented separately.
Performance	Execution time and memory observations are recorded.
Analysis	Stack usage and function-call overhead are explained.
Complexity	Time and space complexity are identified correctly.
Conclusion	A justified practical comparison is provided.

2. Write the Objective

To implement recursive and iterative versions of Merge Sort and experimentally compare them with respect to execution time, memory usage, recursion stack usage, function-call overhead, and practical efficiency.

- To understand the difference between recursive and iterative control flow.
- To measure the performance of both approaches using identical inputs.
- To analyze additional stack memory required by recursive calls.
- To determine the practical trade-off between simplicity and resource efficiency.

3. Document the Required Resources

Hardware Requirements

- Computer or laptop

- Minimum 4 GB RAM recommended
- Modern multi-core processor
- Keyboard and display

Software / Tools

- Python 3.x
- Python standard libraries: time, random, tracemalloc, sys
- No external dataset is required.

Programming Language

Python

IDE / Compiler

- IDLE, VS Code, PyCharm, Jupyter Notebook, or any Python 3-compatible environment.

Dataset / Input Files

No external dataset is required. The program generates random integer arrays automatically. Using generated data makes it easy to repeat the experiment with different input sizes.

4. Provide the Algorithm / Procedure

A. Recursive Merge Sort

1. Receive the input array.
2. If the array contains zero or one element, return because it is already sorted.
3. Find the middle position and divide the array into left and right halves.
4. Recursively apply Merge Sort to the left half.
5. Recursively apply Merge Sort to the right half.
6. Merge the two sorted halves into one sorted array.
7. Return the sorted result.

B. Iterative Merge Sort

8. Receive the input array.
9. Start with subarray width equal to 1.
10. Merge adjacent pairs of subarrays of the current width.
11. Double the width after each complete pass.
12. Continue until the width is greater than or equal to the array size.
13. Return the sorted array.

Procedure for the Experiment

14. Generate one random input array for each selected size.
15. Create two identical copies of the input.
16. Run the recursive implementation on the first copy.
17. Run the iterative implementation on the second copy.
18. Measure execution time using a high-resolution timer.
19. Measure peak Python memory usage using tracemalloc.
20. Verify that both outputs are sorted and identical.
21. Repeat the test if necessary and use representative measurements.
22. Record the observations in a table and compare the two methods.

5. Record the Implementation

Source Code / Program

```
import random
import time
import tracemalloc
import sys

sys.setrecursionlimit(1000000)

def merge(left, right):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

```
def recursive_merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = recursive_merge_sort(arr[:mid])
    right = recursive_merge_sort(arr[mid:])

    return merge(left, right)


def iterative_merge_sort(arr):
    n = len(arr)
    width = 1

    while width < n:
        result = []

        for start in range(0, n, 2 * width):
            mid = min(start + width, n)
            end = min(start + 2 * width, n)

            left = arr[start:mid]
            right = arr[mid:end]

            result.extend(merge(left, right))

        arr = result
        width *= 2

    return arr


def measure(sort_function, data):
    test_data = data.copy()

    tracemalloc.start()
    start = time.perf_counter()

    sorted_data = sort_function(test_data)
```

```
    end = time.perf_counter()
    current, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()

    return sorted_data, end - start, peak / 1024


sizes = [1000, 5000, 10000, 50000, 100000]

print("Merge Sort: Recursive vs Iterative")
print("-" * 72)
print(f"{'Size':>10} {'Recursive Time(s)':>20} {'Iterative Time(s)':>20} "
      f"{'Rec Mem(KB)':>15} {'Iter Mem(KB)':>15}")

for n in sizes:
    data = [random.randint(1, 1000000) for _ in range(n)]

    rec_result, rec_time, rec_mem = measure(
        recursive_merge_sort, data
    )

    itr_result, itr_time, itr_mem = measure(
        iterative_merge_sort, data
    )

    assert rec_result == itr_result
    assert rec_result == sorted(data)

    print(f"{n:>10} {rec_time:>20.6f} {itr_time:>20.6f} "
          f"{rec_mem:>15.2f} {itr_mem:>15.2f}")
```

Important Functions / Modules

Function / Module	Purpose
merge()	Combines two sorted lists into one sorted list.
recursive_merge_sort()	Implements top-down recursive Merge Sort.
iterative_merge_sort()	Implements bottom-up Merge Sort using loops.

measure()	Measures execution time and peak memory.
random	Generates random test data.
time.perf_counter()	Provides high-resolution execution timing.
tracemalloc	Measures Python memory allocation and peak usage.
sys.setrecursionlimit()	Provides sufficient recursion depth for testing.

Significant Code Sections

- The recursive version divides the input until single-element lists are obtained and then merges them.
- The iterative version starts with runs of size 1 and repeatedly doubles the run size.
- Both versions use the same merge operation so the comparison focuses on recursive versus iterative control flow.
- tracemalloc records peak allocated Python memory during each sort.
- assert statements verify that both algorithms produce the same correct result.

Screenshots of Implementation

For the final lab record, insert a screenshot of the source code in the IDE and a screenshot of the program output after running the experiment.

6. Record Input and Output

Test Case 1

Item	Details
Input	[38, 27, 43, 3, 9, 82, 10]
Recursive Output	[3, 9, 10, 27, 38, 43, 82]
Iterative Output	[3, 9, 10, 27, 38, 43, 82]
Result	PASS

Performance Test Cases

Test Case	Input Size	Input Type	Expected Result
TC1	1,000	Random integers	Sorted ascending
TC2	5,000	Random integers	Sorted ascending
TC3	10,000	Random integers	Sorted ascending
TC4	50,000	Random integers	Sorted ascending
TC5	100,000	Random integers	Sorted ascending

Note: The exact execution time and memory values depend on the computer, Python version, background processes, and system load. Therefore, the program should be run on the target system and the measured values should be entered in the result table below.

7. Document the Results

Experimental Observation Table

Input Size	Recursive Time (s)	Iterative Time (s)	Recursive Peak Memory (KB)	Iterative Peak Memory (KB)	Correctness
1,000	Record from run	Record from run	Record from run	Record from run	PASS
5,000	Record from run	Record from run	Record from run	Record from run	PASS
10,000	Record from run	Record from run	Record from run	Record from run	PASS
50,000	Record from run	Record from run	Record from run	Record from run	PASS
100,000	Record from run	Record from run	Record from run	Record from run	PASS

Expected Experimental Trend

- Both implementations should show approximately O(n log n) growth in execution time.
- The recursive implementation creates additional function-call frames during division.
- The iterative implementation avoids recursive call-stack growth.
- The measured memory difference can vary because the Python implementation uses temporary lists and allocations during merging.
- For meaningful comparison, use the same machine and repeat tests under similar system conditions.

Suggested Graphs

- Line graph: Input Size vs Execution Time for Recursive and Iterative Merge Sort.
- Line graph: Input Size vs Peak Memory Usage for Recursive and Iterative Merge Sort.

Insert the graphs after collecting the actual measurements. Use the same input sizes for both lines so the comparison is direct.

8. Analyze the Results

Whether the Expected Output Was Obtained

The expected output is obtained when both implementations produce exactly the same sorted array for every test case. The program verifies this using assertions. The sorting result should be in ascending order.

Performance of the Implemented Methods

Both recursive and iterative Merge Sort have the same asymptotic time complexity, O(n log n). However, they can differ in practical execution time. Recursive Merge Sort performs many function calls, which introduces call overhead and requires stack frames. Iterative Merge Sort uses loops and therefore avoids recursive function-call overhead. As a result, the iterative approach can be faster in practical implementations, although the exact result depends on the programming language and implementation details.

Time Complexity

Operation	Recursive Merge Sort	Iterative Merge Sort
Best Case	O(n log n)	O(n log n)
Average Case	O(n log n)	O(n log n)
Worst Case	O(n log n)	O(n log n)

Space Complexity

The standard Merge Sort algorithm requires O(n) auxiliary memory for merging. In the recursive version, there is also O(log n) recursion-stack usage because the recursion depth is logarithmic. In the iterative version, there is no recursion stack, so it avoids this additional call-stack requirement. In the Python program above, temporary lists created during slicing and merging also contribute to actual peak memory usage.

Stack Usage and Function-Call Overhead

- Recursive approach: approximately O(log n) maximum recursion depth.
- Every recursive division creates a function-call frame until the base cases are reached.
- Function-call creation and return introduce overhead.
- Iterative approach: no recursive call stack is required.
- Loop control generally has lower overhead than repeated recursive calls.
- Therefore, iterative Merge Sort is usually preferable when minimizing call-stack usage is important.

Advantages and Limitations

Approach	Advantages	Limitations

Recursive	Simple divide-and-conquer structure; easy to understand; closely matches the algorithm definition.	Uses recursion stack; has function-call overhead; very deep recursion can be a concern in some languages.
Iterative	Avoids recursion; no recursion-stack growth; can reduce function-call overhead; suitable for large inputs.	Control logic can be less intuitive; implementation is slightly more complex to understand initially.

Comparison with Alternative Approaches

Approach	Typical Average Time	Extra Space	Main Characteristic
Recursive Merge Sort	O(n log n)	O(n) + O(log n) stack	Simple recursive structure
Iterative Merge Sort	O(n log n)	O(n) auxiliary	No recursion stack
Quick Sort	O(n log n)	Depends on implementation	Often fast in practice; worst case $O(n^2)$
Heap Sort	O(n log n)	O(1) auxiliary	In-place; guaranteed O(n log n)

Interpretation

The experiment demonstrates that changing recursion to iteration does not change the asymptotic O(n log n) running time of Merge Sort. The main difference is the way control information is stored. Recursive Merge Sort stores pending work in the call stack, while iterative Merge Sort manages the same progression explicitly through loop variables and subarray widths. For large inputs, avoiding recursive calls can reduce overhead and eliminate recursion-stack growth. Therefore, when practical execution efficiency, predictable stack usage, and robustness for large inputs are priorities, the iterative approach is generally the better choice. Recursive Merge Sort remains attractive for clarity and teaching because its code directly represents the divide-and-conquer idea.

9. Write the Conclusion

Both recursive and iterative Merge Sort implementations correctly sort the input and have the same asymptotic time complexity of O(n log n). The recursive version is easier

to express and understand, but it requires O(log n) recursion-stack space and incurs function-call overhead. The iterative version avoids recursive calls and stack growth while maintaining O(n log n) time complexity. The experimental measurements should be used to confirm the performance difference on the target computer. Overall, the iterative implementation is generally more efficient for practical applications where lower function-call overhead and predictable stack usage are important, while the recursive implementation is often preferred for simplicity and readability.

Final Result

The recursive and iterative Merge Sort methods were implemented and compared. Both methods produce correct sorted output. The analysis shows that iterative Merge Sort has a practical advantage by eliminating recursion-stack usage and recursive function-call overhead, while both retain the O(n log n) time complexity of Merge Sort.